

Python Power-Ups Placement Handbook

Chapter 1 (Part 2): Advanced Comprehensions, Lambda & LEGB

Conditional Comprehensions

Syntax: `[x if condition else y for item in iterable]`. Useful when every item should produce a value.

Syntax: `[x if condition else y for item in iterable]`. Useful when every item should produce a value.

Syntax: `[x if condition else y for item in iterable]`. Useful when every item should produce a value.

Syntax: `[x if condition else y for item in iterable]`. Useful when every item should produce a value.

Nested Comprehensions

Multiple for-loops may be written inside a comprehension. Read them left to right exactly like nested loops.

Multiple for-loops may be written inside a comprehension. Read them left to right exactly like nested loops.

Multiple for-loops may be written inside a comprehension. Read them left to right exactly like nested loops.

Multiple for-loops may be written inside a comprehension. Read them left to right exactly like nested loops.

Dictionary Inversion

Example: `{v:k for k,v in data.items()}`. Useful for reversing mappings when values are unique.

Example: `{v:k for k,v in data.items()}`. Useful for reversing mappings when values are unique.

Example: `{v:k for k,v in data.items()}`. Useful for reversing mappings when values are unique.

Example: `{v:k for k,v in data.items()}`. Useful for reversing mappings when values are unique.

Real-world Examples

Cleaning datasets, filtering API responses, generating reports, transforming CSV rows, and preparing ML features.

Cleaning datasets, filtering API responses, generating reports, transforming CSV rows, and preparing ML features.

Cleaning datasets, filtering API responses, generating reports, transforming CSV rows, and preparing ML features.

Cleaning datasets, filtering API responses, generating reports, transforming CSV rows, and preparing ML features.

Performance

Comprehensions are often faster than repeatedly calling `append()`, but readability is more important than micro-optimizations.

Comprehensions are often faster than repeatedly calling `append()`, but readability is more important than micro-optimizations.

Comprehensions are often faster than repeatedly calling `append()`, but readability is more important than micro-optimizations.

Comprehensions are often faster than repeatedly calling `append()`, but readability is more important than micro-optimizations.

Lambda with map/filter/sorted

`map(func,data), filter(func,data), sorted(data,key=lambda x:x[1])`. Prefer comprehensions over `map/filter` when they improve readability.

`map(func,data), filter(func,data), sorted(data,key=lambda x:x[1])`. Prefer comprehensions over `map/filter` when they improve readability.

`map(func,data), filter(func,data), sorted(data,key=lambda x:x[1])`. Prefer comprehensions over `map/filter` when they improve readability.

`map(func,data), filter(func,data), sorted(data,key=lambda x:x[1])`. Prefer comprehensions over `map/filter` when they improve readability.

LEGB Deep Dive

Local variables shadow enclosing variables. `global` modifies a module variable. `nonlocal` modifies the nearest enclosing function variable.

Local variables shadow enclosing variables. `global` modifies a module variable. `nonlocal` modifies the nearest enclosing function variable.

Local variables shadow enclosing variables. `global` modifies a module variable. `nonlocal` modifies the nearest enclosing function variable.

Local variables shadow enclosing variables. `global` modifies a module variable. `nonlocal` modifies the nearest enclosing function variable.

Interview Problems

Predict outputs involving variable scope, closures, lambda capture, and nested comprehensions.

Predict outputs involving variable scope, closures, lambda capture, and nested comprehensions.

Predict outputs involving variable scope, closures, lambda capture, and nested comprehensions.

Predict outputs involving variable scope, closures, lambda capture, and nested comprehensions.

Best Practices

Keep comprehensions simple. Avoid side effects. Use `def` instead of `lambda` for complex logic.

Keep comprehensions simple. Avoid side effects. Use `def` instead of `lambda` for complex logic.

Keep comprehensions simple. Avoid side effects. Use `def` instead of `lambda` for complex logic.

Keep comprehensions simple. Avoid side effects. Use `def` instead of `lambda` for complex logic.

Quick Revision Sheet

List: `[expr for x in data if cond]`

Dict: `{k:v for x in data}`

Set: `{expr for x in data}`

Generator: `(expr for x in data)`

Lambda: `lambda args: expr`

LEGB: Local → Enclosing → Global → Built-in